

IA Agentic

Lang chain - Llama Index

SOMMAIRE

FONDAMENTAUX.....	4
Introduction.....	4
Veille technologique.....	4
Pratique.....	4
étape à suivre:.....	4
Exercices:.....	4
CHATBOT.....	5
Introduction.....	5
Pratique.....	5
Exercice 1 Stateless.....	5
Exercice 2 Stateful.....	5
Note Groq.....	6
CHATBOT LANG CHAIN.....	7
Introduction.....	7
Pratique.....	7
Concepts introduits :.....	7
La chaîne, le concept de LCEL:.....	8
Communication avec le llm.....	8
OPTIMISATION DE LA MÉMOIRE.....	9
Le comptage des tokens.....	9
Veille technologique.....	9
Stratégie d'optimisation.....	9
Stratégies Standard.....	9
Stratégies Exotiques (Niveau Expert).....	10
Exercices.....	10
PERSISTANCE ET BASES VECTORIELLES.....	11
Introduction.....	11
Initiation à Chroma DB (local).....	11
installation.....	11
utilisation.....	11
Exercice.....	12
Si la base de données est distante.....	12
PG Vector.....	13
installation.....	13
utilisation.....	13

Note.....	13
RAG.....	14
Introduction.....	14
Concepts fondamentaux.....	14
RAG avec LangChain & ChromaDB.....	15
Installation.....	15
Utilisation.....	15
définition du pipe.....	16
Exercice.....	17
RAG avec Llama Index et FAISS.....	17
Installation.....	17
Utilisation.....	17
Exercice.....	18
Cerise sur le gâteau.....	18
AGENTS ET OUTILS.....	19
Introduction.....	19
Concepts fondamentaux : La boucle ReAct (Reason + Act).....	19
Créer un Tool personnalisé.....	20
Donner les tools au llm.....	20
Définir l'agent.....	20
Création du Graphe.....	21
Poser la question, l'invocation!.....	22
Visualiser le graphe.....	23
Noeuds d'évaluation.....	24
Introduction.....	24
Schéma logique (graphe).....	25
Mise en pratique.....	26
Exercice.....	28
ORCHESTRATION MULTI-AGENTS.....	29
Pratique.....	29

FONDAMENTAUX

L'autonomie

Concepts : Tokenization, Température, Context Window.

Objectif : Comprendre que l'IA est un moteur qu'on peut posséder.

Introduction

Le but de ce module est de démythifier le fonctionnement des Large Language Models (LLM) en opérant une transition de l'usage "consommateur" (SaaS) vers l'usage "ingénieur" (Local).

Veille technologique

Qu'est ce que la **Tokenization**, la **Température**, et la **Context Window**

Pratique

Installation d'**Ollama** et premier script Python pour discuter avec un modèle local.

étape à suivre:

- Aller sur <https://ollama.com/> et installer Ollama en local
- Dans votre terminale exécuter : `ollama run llama3.2:3b`
- `qwen3:4B / 8B` ou `gemma4:e4b`
- Créer un notebook et `import ollama` et découvrir la fonction `ollama.generate`

Exercices:

Température : En utilisant le modèle 3B demandez à Ollama où est assis le chat? Testez avec une température nulle et une température égale à un.

Mémoire : Dites à Ollama comment vous vous appelez, puis demandez-lui votre nom?

Contexte Window : préparez un mot de passe dans une phrase (prompt 1) et demandez-lui quel est le mot de passe. Créez un prompt 2 (2000*"Lorem Ipsum") et donnez lui prompt 1 + prompt 2 et demandez-lui quel est le mot de passe.

CHATBOT

La mémoire

Concepts : Windows Memory, Buffer Memory, Stateless,.

Objectif : Avoir une discussion sans amnésie.

Introduction

L'objectif pédagogique est d'apprendre à concevoir une couche logicielle capable de simuler une continuité conversationnelle. Nous allons voir comment un développeur peut "donner de la mémoire" à un modèle en gérant manuellement un tampon de messages (Buffer) qui est réinjecté dans la Context Window à chaque tour d'inférence.

Pratique

Exercice 1 Stateless

Création d'une fonction "chatbot()" dans un script python dans lequel on utilisera une boucle while pour communiquer (`user_input = input("\nClient : ")`).

Si l'utilisateur utilise les mots **exit**, **stop** ou **fin** la conversation s'arrête. On communique avec ollama en utilisant la même fonction qu'avant (`ollama.generate`).

Exercice 2 Stateful

On va maintenant créer une deuxième fonction "chatbot_memory()" qui aura les mêmes critères d'arrêt que précédemment. La mémoire va se conserver de message en message en empilant les messages dans une liste de dictionnaire. Le message initial est:

```
messages = [
    {
        'role': 'system',
        'content': 'Tu es un assistant technique professionnel. Tu dois aider le
client en te basant sur les informations fournies dans la discussion.'
    }
]
```

On empile les messages de l'utilisateur

```
messages.append({'role': 'user', 'content': user_input})
```

Mais aussi les messages de l'assistant

```
messages.append({'role': 'assistant', 'content': assistant_reply})
```

La fonction d'ollama `ollama.generate` prend une chaîne de caractère en entrée et non un dictionnaire, il serait toutefois possible de lui donner une très longue chaîne de caractère en précisant les rôles dans laquelle on ajouterait manuellement les rôles pour savoir qui à dit quoi, c'est ce qu'on faisait autrefois (préhistoire).

Pour la gestion d'un dictionnaire il y a la fonction `ollama.chat`

Note Groq

On utilise Ollama mais on peut utiliser n'importe quel LLM, notamment les modèles hébergés par Groq sont très performant et le nombre de tokens gratuit est largement suffisant pour les utiliser en formation.

Dans un fichier `.env` :

```
GROQ_API_KEY=gsk_rXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Dans le code :

```
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()

llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0.7)
```

Pour utiliser l'énorme 120B de GPT

CHATBOT LANG CHAIN

L'abstraction

Introduction

Le but de cette partie est d'industrialiser notre chatbot. LangChain ne manipule pas des dictionnaires bruts, mais des objets Python typés. Cette abstraction permet de changer de modèle (passer d'Ollama à OpenAI par exemple) sans changer la structure de notre code. De plus, cela nous permet de commencer à utiliser les outils de Lang Chain, bibliothèque que l'on utilisera également pour les agents IA. On va donc refaire le même exercice mais en transformant cette fois ci les dictionnaires en objet Python.

Pratique

Dans cette version de l'exercice, nous abandonnons les dictionnaires Python classiques pour utiliser des Objets de Messages. Cette approche est la norme industrielle car elle permet de séparer la structure logique du message de son contenu textuel.

Concepts introduits :

- **HumanMessage & AIMessage** : Ce sont des classes spécifiques qui typent l'origine du message. Utiliser ces objets permet à LangChain de formater automatiquement la discussion selon le modèle choisi (Ollama, OpenAI, etc.), garantissant une portabilité totale de votre code.
- **ChatPromptTemplate** : Contrairement à une simple chaîne de caractères, ce template permet de définir une structure rigide et réutilisable pour vos prompts, facilitant la gestion des consignes système et des entrées utilisateurs.
- **MessagesPlaceholder** : C'est l'outil clé pour la gestion de la mémoire. Il agit comme un "réservoir" dynamique à l'intérieur de votre prompt. Il indique à LangChain exactement où injecter l'historique des échanges accumulés, sans avoir à manipuler manuellement des listes ou des concaténations complexes.

Le message de départ devient:

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "Tu es un assistant technique pro. Réponds de manière concise."),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}"),
])
```

En utilisant ces objets, vous ne construisez plus un script pour un modèle précis, vous construisez une architecture de communication capable de piloter n'importe quelle IA du marché.

La chaîne, le concept de LCEL:

Le Pipe, opérateur “|” (barre verticale et non un “l” ou “i” majuscule) est le cœur de la syntaxe moderne de LangChain, appelée LCEL (LangChain Expression Language). Au lieu d'écrire des fonctions imbriquées complexes comme **model(prompt(input))**, on utilise le pipe pour créer une séquence logique où la sortie de l'un devient l'entrée du suivant.

Pour l'exercice on utilisera la chaîne suivante :

```
chain = prompt | llm
```

où (à importer)

```
llm = ChatOllama(model="llama3.2:3b", temperature=0)
```

Cela indique que le prompt rentre ensuite dans le llm. C'est un outils complexe qui permet de gérer des prompts par batch (`chain.batch([q1, q2, ...])`) ou d'utiliser des modèles de secours (`chain = (prompt | llm).with_fallbacks([backup_llm])`) et bien d'autres choses.

Communication avec le llm

Le modèle ne sera plus directement la fonction d'ollama mais une fonction de lang chain :

```
response = chain.invoke({
    "input": user_input,
    "chat_history": history
})
```

OPTIMISATION DE LA MÉMOIRE

Le coûts

Le comptage des tokens

Avant d'optimiser, il faut savoir mesurer. Un token n'est ni un mot, ni un caractère.

```
nb_tokens = llm.get_num_tokens(texte)
```

Chaque token a un prix, qu'il soit facturé à la requête (Cloud) ou en électricité/temps de calcul (Local). Plus l'historique est long, plus le GPU doit "lire" de données avant de générer le premier mot. Un contexte de 128k tokens nécessite beaucoup plus de mémoire vidéo qu'un contexte de 4k. Si la VRAM sature, le calcul bascule sur la RAM (CPU), et la vitesse chute de 90%.

Veille technologique

Créez un tableau de comparaison de prix au millions de tokens en fonctions des modèles

Stratégie d'optimisation

Stratégies Standard

- **Full Buffer** : On envoie tout. Précision à 100%, coût maximum.
- **Sliding Window** : On garde les **k** derniers messages. Simple, mais amnésique.
- **Summary Buffer** : On résume tout le passé en un paragraphe. Économique, mais perd les détails.

```
nouveau_resume = llm.invoke(f"Update ce résumé : '{ancien_resume}' avec  
cette nouvelle info : '{message_sortant}'")
```

- **Moving Summary** : On conserve uniquement les **k** derniers messages en clair et on résume tout ce qui suit.
- **Hierarchical (Incremental)** : On conserve les **k** derniers messages et on résume par blocs de 10 messages. Très bon équilibre. On conserve plusieurs blocs.

Stratégies Exotiques (Niveau Expert)

- **Entity Memory** : L'IA extrait uniquement les faits (nom, projet, préférences) dans une "fiche client" et oublie le reste.
- **Vectorized History** : On transforme les vieux messages en vecteurs. On ne ressort que les messages du passé qui ressemblent à la question actuelle.
- **Context Re-ranking** : On prend les derniers messages et on supprime tout ce qui est inutile (politesse, répétitions) avant l'envoi.

Exercices

- Implémentation de la stratégie de la **Sliding Window** et comparaison des résultats. Constatez que les informations du premier message sont oubliées après **k** messages.
- Implémentez la stratégie du **Summary Buffer** et comparez les résultats après **k** messages. Est ce que l'information importante est conservée?
- Implémenter la stratégie de la **Moving Summary** et comparer les résultats par rapport au message précédent.

PERSISTANCE ET BASES VECTORIELLES

La mémoire

Concepts : Embeddings, Vector Stores, Chunks (découpage).

Objectif : Avoir une mémoire en dur.

Introduction

Nos stratégies précédentes (Window, Summary) fonctionnent en mémoire vive. Si le script s'arrête, l'IA oublie tout. On utilisera une base de données vectorielle comme Chroma DB pour stocker les échanges.

Initiation à Chroma DB (local)

installation

```
pip install chromadb langchain-chroma
```

utilisation

On aura besoin aussi des embedding pour transformer le chat en données vectorielles

```
from langchain_ollama import ChatOllama, OllamaEmbeddings
model_name = "llama3.2:3b"
llm = ChatOllama(model=model_name, temperature=0)
embeddings = OllamaEmbeddings(model=model_name)
```

On devra définir l'endroit où la base de données local est stockées et définir le vecteur store:

```
DB_PATH = "./ma_memoire_profonde"
vectorstore = Chroma(
    collection_name="historique_global",
    embedding_function=embeddings,
    persist_directory=DB_PATH
)
```

Afin que le modèle soit capable d'utiliser à la fois la mémoire à court terme (history) et les données de la base de données il faut lui passer les deux. Cependant on ne va pas lui donner toute la base de données, on va lui donner les contenus qui sont similaires à la discussion actuelle.

```
docs_du_passe = vectorstore.similarity_search(user_input, k=3)
context_archives = "\n".join([doc.page_content for doc in docs_du_passe])

response = chain.invoke({
    "input": user_input,
    "session_history": history_session,
    "archives": context_archives
})
```

Il va falloir sauvegarder le chat dans le store au fur et à mesure. C'est un peu au choix, juste les questions, juste les réponses ou les deux

```
echange_formate = f"L'utilisateur a demandé : {user_input} | L'assistant a répondu : {response.content}"

vectorstore.add_texts(texts=[echange_formate])
```

Exercice

Implémentez la base de données chroma DB et utilisez une des stratégie d'optimisation du chapitre précédent. Pour vérifier que la sauvegarde fonctionne, terminez votre code puis relancez le, vérifier de quoi le llm se rappelle.

Si la base de données est distante

```
vectorstore = Chroma(
    collection_name="historique_global",
    embedding_function=embeddings,
    persist_directory=DB_PATH
)
```

le DB_PATH de persist_directory est remplacé les informations réseau

```
pip install chromadb
```

```
import chromadb
persistent_client = chromadb.HttpClient(host="localhost", port=8000)
```

PG Vector

PostgreSQL possède une extension pour les bases de données vectorielles : PGVector. L'avantage est qu'on peut mélanger des données classiques (Nom, Date, ID) et des vecteurs dans la même table. Par exemple :

Trouve-moi les documents similaires à 'Facture' (Vecteur) MAIS seulement ceux créés par l'utilisateur ID 42 il y a moins de 30 jours (SQL).

Ceci est très dur à faire avec Chroma, mais trivial avec PGVector.

installation

```
pip install pgvector psycopg2-binary
```

utilisation

```
from langchain_community.vectorstores import PGVector

# La chaîne de connexion (Connection String)
CONNECTION_STRING =
"postgresql+psycopg2://postgres:password@localhost:5432/postgres"

# Initialisation du vectorstore

vectorstore = PGVector(
    connection_string=CONNECTION_STRING,
    collection_name="historique_assistant",
    embedding_function=embeddings, # Ton OllamaEmbeddings
    use_jsonb=True # Optimisation pour PostgreSQL
)
```

Encore l'avantage d'utiliser Lang Chain est qu'on ne change que le vecteur store, Lang Chain se charge du reste.

Note

On verra FAISS (Facebook AI Similarity Search) plus tard. Ce n'est pas une base de données (ChromaDB ou Postgres) mais une bibliothèque pour la recherche de vecteurs dans la RAM.

RAG

Le savoir

Concepts : Embeddings, Vector Stores, Chunks (découpage).

Objectif : Arrêter les hallucinations en forçant l'IA à lire des sources fiables.

Introduction

Le savoir des LLM remonte à leur date d'entraînement, ils ne sont pas connectés à internet et n'ont pas de mémoire. Dans le chapitre précédent nous avons vu comment gérer la mémoire, les tokens et la persistance des connaissances en général. De plus, un LLM cherche toujours à vous donner une réponse et on a tous remarqué qu'un LLM pouvait halluciner. Pour pallier ces défauts nous allons à présent nous intéresser aux RAG (retrieval-augmented generation) qui fonctionnent en utilisant une base de connaissance. On peut dire que c'est très similaire de ce qu'on a fait avec les chatbot où la base de connaissance était la discussion elle-même.

Concepts fondamentaux

Le RAG suit toujours ces 5 étapes :

- **Ingestion** : Lire la source de données.
- **Chunking** : Découper le texte (car on ne peut pas tout envoyer d'un coup).
- **Embedding** : Transformer le texte en coordonnées mathématiques (vecteurs).
- **Retrieval** : Chercher les morceaux les plus proches de la question.
- **Augmentation** : Injecter ces morceaux dans le prompt final.

Petite veille technologique sur les méthodes de Chunking :

- Character Split
- Recursive Character
- Document Based
- Semantic Chunking

RAG avec LangChain & ChromaDB

Installation

Pour cette partie, nous avons besoin des briques LangChain pour le chargement, le découpage et la gestion de Chroma DB.

```
pip install      langchain-community
                  langchain-text-splitters
                  langchain-chroma
                  langchain-ollama
```

Utilisation

On utilisera *TextLoader* pour charger un document texte et *RecursiveCharacterTextSplitter* pour le découpage des chunks, le texte sera découpé en utilisant la méthode *split_documents*.

```
loader = TextLoader("./menu.txt", encoding="utf-8")
documents = loader.load()

# Découpage intelligent (Recursive)
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=["\n\n", "\n", " ", ""]
)
chunks = text_splitter.split_documents(documents)
```

On utilisera l'embedding lié au modèle ici Llama3.2:3B . Attention, on ne peut pas changer de modèle d'embedding sans tout réindexer. Puis, la base de données Chroma DB reçoit le document en utilisant la fonction *from_documents* et on crée le **retriever** avec la méthode *as_retriever* sur le vecteur store.

```
embeddings = OllamaEmbeddings(model="llama3.2:3b")

vectorstore = Chroma.from_documents(documents=chunks, embedding=embeddings)

retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```


Puis on doit définir le template de réponse du RAG, sa personnalité en quelque sorte et comment il comprends les questions

```
template = """Tu es un .... Réponds à la question du client en utilisant
uniquement les data fournies ci-dessous.
Si ..., réponds poliment que tu ne sais pas.
data (Contexte) :
{context}
Question client : {question}
"""
prompt = ChatPromptTemplate.from_template(template)
```

définition du pipe

Comme on l'a vu précédemment et dans la version actuelle (2026) de Lang Chain toute la logique se code en définissant le pipe.

Pour que le prompt fonctionne il lui faut un dictionnaire contenant un *contexte* et la *question*. On utilise la fonction *RunnablePassthrough* pour lui dire de ne pas toucher à la question et la laisser tel quel.

Le contexte sera le résultat du **retriever**, les informations les plus proches en base de données, cependant comme on a k lignes de réponse on doit les regrouper en un seul texte, on fera donc un pipe entre le retriever et une fonction perso pour générer. La fonction a besoin de la sortie du **retriever** alors le pipe s'écrit `retriever | fonction`.

Maintenant on peut donc faire le pipe entre le prompt et le dictionnaire d'entrée, puis le pipe pour tout donner au llm et enfin formater la réponse (dernier pipe) avec *StrOutputParser* qui enlève toutes les métadonnées pour ne garder que la réponse pure.

```
def format_docs(docs):
    return "\n\n".join([d.page_content for d in docs])
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```

Finalement il n'y a plus qu'à lui poser la question et analyser la réponse :

```
response = rag_chain.invoke(question_test)
```

Exercice

Vous allez faire un service de chatbot qui renseignera les clients sur le contenu du menu.

Le fichier menu.txt vous sera fourni et le chat bot devra se comporter comme un serveur virtuel, s'il ne connaît pas la réponse il doit rester poli.

En utilisant LangChain et la structure LCEL (les pipes |), développez un script qui :

- Charge le fichier menu.txt.
- Découpe le texte de manière intelligente (utilisez *RecursiveCharacterTextSplitter*).
- Stocke les informations dans une base vectorielle *ChromaDB*.
- Interroge le modèle Llama 3.2 via Ollama en lui fournissant les morceaux du menu les plus pertinents (k=3).

Jouez avec les paramètres, changez les méthodes de chunks. Notez tout ce que vous trouvez anormal.

RAG avec Llama Index et FAISS

Installation

```
pip install llama-index
             llama-index-llms-ollama
             llama-index-embeddings-ollama
             llama-index-vector-stores-faiss
             faiss-cpu
             pypdf
```

Utilisation

On doit mettre toutes les données dans un dossier et Llama index va tout gérer, même si ils sont de types différents

```
documents = SimpleDirectoryReader("./data").load_data()
```

Puis on configure FAISS pour les accès rapide en mémoire

```
# Dimension 3072 pour Llama 3.2

d = 3072
faiss_index = faiss.IndexFlatL2(d)
vector_store = FaissVectorStore(faiss_index=faiss_index)
storage_context = StorageContext.from_defaults(vector_store=vector_store)
```

La dimension est très importante, une mauvaise dimension donnerait une erreur. La dimension dépend du modèle d'embedding. Pour Llama 3.2, c'est 3072. Si vous changez de modèle et que vous avez un message d'erreur 'Dimension mismatch', c'est ici qu'il faudra ajuster.

Puis on crée le moteur de recherche

```
index = VectorStoreIndex.from_documents(documents, storage_context=storage_context)
query_engine = index.as_query_engine()
```

et il ne reste plus qu'à lui poser la question

```
response = query_engine.query("question?")
```

On remarque beaucoup de différence avec Lang Chain, le code est beaucoup plus compact, par contre il est beaucoup moins modulaire (pas d'agent directement). Llama Index utilise pypdf en arrière-plan pour extraire proprement le texte des fichiers PDF sans que vous ayez à coder l'extracteur.

Exercice

Création d'un chatbot multisupport (markdown, text, pdf) sur le référentiel de compétence de développeur en intelligence artificielle Microsoft by Simplon. Les fichiers vous sont fournis et vous devez les indexer et questionner votre RAG!

Cerise sur le gâteau

Si vous voulez juste faire lire un PDF à une IA, Llama Index suffit. Si vous voulez créer un agent autonome qui discute, réfléchit et agit, LangChain est idéal. Mais pour les projets d'envergure, on utilise Llama Index pour trouver la bonne info et on la donne à LangChain pour qu'il sache quoi en faire.

AGENTS ET OUTILS

L'action

Concepts : Tools (Outils), LLM Reasoning (Raisonnement), Boucle ReAct.

Objectif : Transformer le LLM d'un simple "parleur" en un "acteur" capable d'interagir avec le monde réel (calculs, recherches, fichiers).

Introduction

Jusqu'à présent, nos IA étaient "fermées" : elles ne connaissaient que ce qu'elles avaient appris en 2024 ou ce qu'on leur donnait dans un menu. Un **Agent**, c'est un LLM doté d'une **boîte à outils**. Au lieu de répondre immédiatement, l'agent va :

- **Réfléchir** à la question.
- **Choisir** l'outil le plus adapté.
- **Utiliser** l'outil et observer le résultat.
- **Répondre** (ou choisir un autre outil si nécessaire).

En plus d'avoir une tête, l'agent a des bras et peut effectuer des actions en toute autonomie.

Concepts fondamentaux : La boucle ReAct (Reason + Act)

C'est le mode de pensée standard des agents. Le LLM génère une séquence :

- **Thought (Pensée)** : "Je dois trouver la météo à Paris, je vais utiliser l'outil de recherche."
- **Action** : L'appel technique à l'outil.
- **Observation** : Le résultat renvoyé par l'outil (ex: "15°C, Pluie").
- *(Répétition si besoin)* → **Final Answer**.

Créer un Tool personnalisé

Avant d'utiliser des outils complexes, il faut comprendre comment on "explique" un outil à une IA. On utilise des **Python Docstrings** pour décrire l'outil au LLM.

Exemple d'un outil de calcul :

```
from langchain_core.tools import tool

@tool
def calculateur_tva(prix_ht: float) -> float:
    """Calcule le prix TTC à partir du prix HT en ajoutant une TVA de 20%.
    Utile pour les calculs financiers précis."""
    return prix_ht * 1.20
```

La docstring `"""xxx"""` sera lu par l'agent ce qui lui permettra de savoir ce que fait l'outil.

Un tool peut être n'importe quel outil que ce soit un moteur de recherche comme `DuckDuckGoSearchRun`, une API en général ou une fonction personnalisée.

Donner les tools au llm

Le Llm a besoin de connaître les tools, sinon il ne sera pas capable de donner l'ordre de les utiliser. On lui donne donc une liste d'outil:

```
tools = [DuckDuckGoSearchRun(), calculateur_tva]
llm_with_tools = llm.bind_tools(tools)
```

Définir l'agent

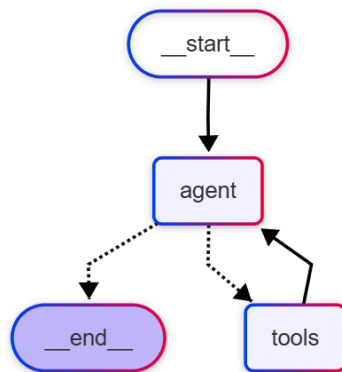
Pour respecter la norme de Lang Chain et de ses Pipes l'agent doit pouvoir recevoir un `MessagesState` et de retourner un message. Toutes les communications dans le Graphe se font avec ces `MessagesState`.

```
def node_agent(state: MessagesState):
    # Il regarde l'historique des messages (state) et décide quoi faire
    response = llm_with_tools.invoke(state["messages"])
    # On renvoie sa réponse (qui s'ajoute à la liste des messages)
    return {"messages": [response]}
```

Création du Graphe

Pour que le système fonctionne il faut à présent construire le graphe d'état. Dans notre logique simple voici la logique:

- l'agent reçoit le message
- il y a une décision conditionnelle, dois-je utiliser les outils?
 - Oui, l'outil est utilisé et le message retourne vers l'agent
 - Non, le travail est terminé et on sort de la boucle



En terme de code l'agent et les tools sont ce qui s'appelle des nodes car il y a une action qui est effectuée. L'agent est le cerveau et les tools sont les bras.

Les connexions entre les nodes sont des edges et il en existe deux type :

- edge : connexion directe entre deux nodes
- conditional_edges : pour exprimer les conditions

Pour coder le graphe on commence par l'initialiser:

```
# MessagesState gère automatiquement la liste des messages (mémoire)
workflow = StateGraph(MessagesState)
```

Puis on crée les deux nodes (cerveaux et bras), *ToolNode* est une brique toute prête qui exécute l'outil.

```
workflow.add_node("agent", node_agent)
workflow.add_node("tools", ToolNode(tools))
```

Enfin on définit les connexions comme dans le graphe ci dessus, on commence par créer le START

```
workflow.add_edge(START, "agent")
```

puis la condition

```
workflow.add_conditional_edges("agent", tools_condition)
```

et on câble le retour des outils vers l'agent

```
workflow.add_edge("tools", "agent")
```

On a pas besoin de câbler le **END** car la fonction *tools_condition* est une fonction "magique" fournie par Lang Graph. À l'intérieur de son code, elle fait exactement ceci :

- Si le dernier message contient un appel d'outil → elle renvoie la chaîne "tools".
- SINON (si l'agent a fini de parler) → elle renvoie la constante END.

Un fois le graphe créé on peut le compiler

```
agent_app = workflow.compile()
```

Poser la question, l'invocation!

```
system_prompt = SystemMessage( content="""Tu es un assistant qui DOIT
utiliser des outils.Pour chaque question, tu DOIS utiliser un outil de
recherche ou de calcul. N'utilise JAMAIS tes propres connaissances pour
des faits ou des prix. Si tu ne trouves pas l'info via l'outil, dis que
tu ne sais pas.""")

query = "Quel est le prix actuel de l'action NVIDIA en temps réel ? Et
si j'en achète 10, combien ça fait TTC avec ton calcul ?"

inputs = {"messages": [system_prompt, HumanMessage(content=query)]}
response = agent_app.invoke(inputs)

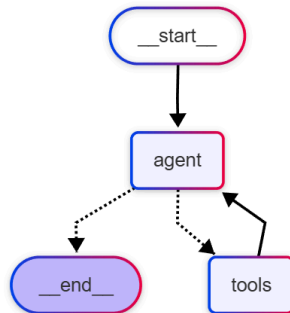
print(f"RÉPONSE : {response['messages'][-1].content}")
```

Visualiser le graphe

- En utilisant mermaid

```
# On récupère le code texte au format Mermaid
mermaid_code = agent_app.get_graph().draw_mermaid()

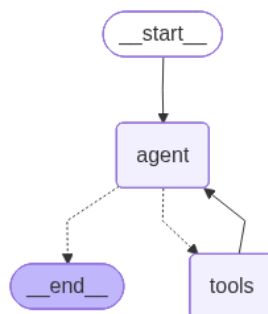
# On l'écrit dans un fichier .mmd
with open("graph.mmd", "w", encoding="utf-8") as f:
    f.write(mermaid_code)
```



- en utilisant graphviz ou pygraphviz (à installer dans le venv)

```
graph_image = agent_app.get_graph().draw_mermaid_png()

with open("mon_graphe.png", "wb") as f:
    f.write(graph_image)
print("✅ Graphe enregistré sous 'mon_graphe.png'")
```



- Lang Graph Studio (Le Graal)

<https://www.blog.langchain.com/langgraph-studio-the-first-agent-ide/>

Noeuds d'évaluation

Le juge

Concepts : Boucle d'évaluation, Critique constructive, Audit de données.

Objectif : Transformer le LLM d'un simple "parleur" en un "acteur" fiable, capable de valider ses propres actions avant de les livrer à l'utilisateur.

Introduction

Si les outils donnent des "bras" à notre IA, le nœud d'évaluation lui donne une **conscience professionnelle**. Jusqu'à présent, notre agent pouvait agir, mais il pouvait aussi se tromper, sauter une étape ou mal interpréter un résultat.

Avec le **Nœud d'Évaluation**, dès que l'agent considère qu'il a fini, son message est transmis au **Juge** qui décide si :

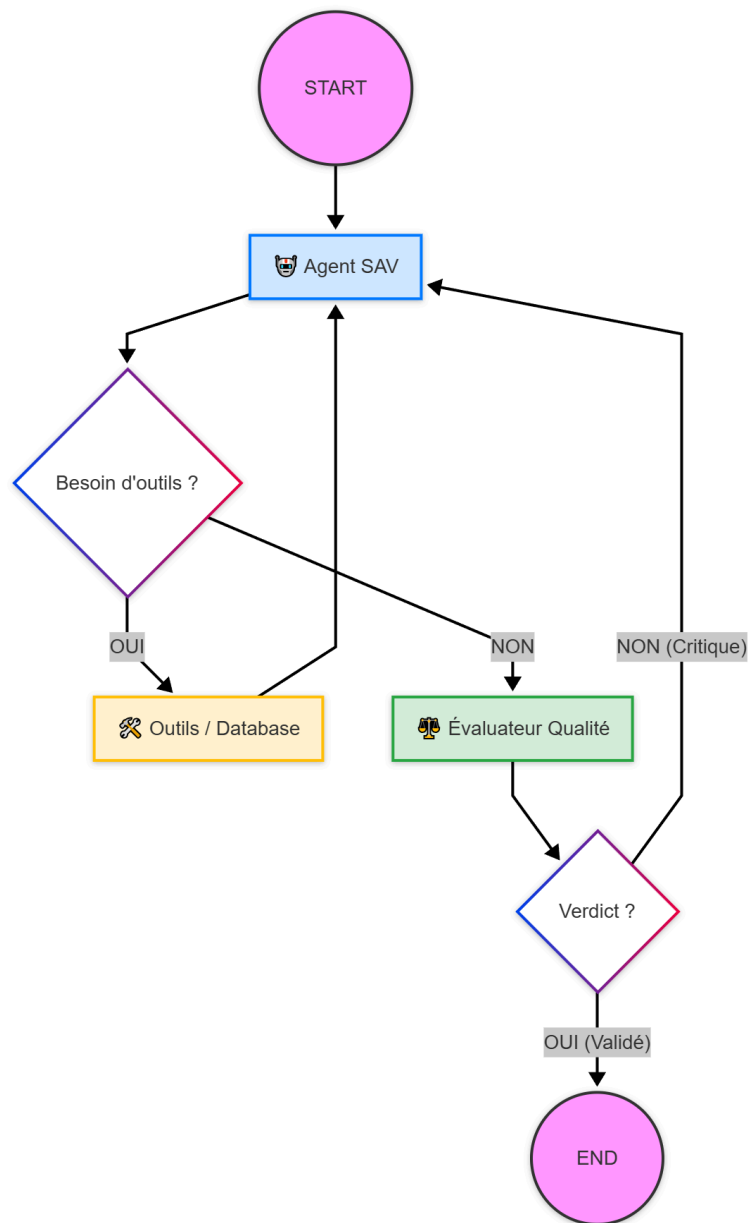
- **Le travail est bien fait** : L'agent a été rigoureux, on sort de la boucle pour répondre à l'utilisateur.
- **Le travail est incomplet** : L'agent a ignoré les outils ou les règles, le juge le renvoie alors en boucle pour correction.

Le **Nœud d'Évaluation** agit comme un superviseur, il va :

- Auditer les preuves
- Vérifier la logique
- Sanctionner ou Valider

En plus d'avoir des bras pour agir, l'agent a désormais un **contrôleur qualité** qui garantit qu'il n'utilise pas sa force n'importe comment.

Schéma logique (graphe)



Mise en pratique

En plus de définir les outils il faudra implémenter le noeud d'évaluation et les conditions pour sortir ou continuer à boucler.

Le graphe se codera de la manière suivante:

```
workflow = StateGraph(MyGraphState)

workflow.add_node("agent", call_model)
workflow.add_node("tools", tool_node)
workflow.add_node("evaluator", quality_control_node)

workflow.add_edge(START, "agent")
workflow.add_conditional_edges("agent", should_continue)
workflow.add_edge("tools", "agent")
workflow.add_conditional_edges("evaluator", route_after_eval)

app = workflow.compile()
```

call_model est l'agent principal avec le prompt système, on détaillera le contenu du prompt plus tard

tool_node se définit comme avant:

```
tools = [...]
tool_node = ToolNode(tools)
```

quality_control_node est notre juge, c'est un LLM avec son propre prompt, ses propres règles qui propagera le verdict à travers le **MessageState**. Comme le message state ne contient pas, par défaut, le champ de décision nous allons créer une classe **MyGraphState** qui héritera de **MessageState** :

```
class MyGraphState(MessagesState):
    verdict: EvaluationVerdict
```

où **EvaluationVerdict** est une classe qui hérite du **BaseModel** de Pydantic pour la conformité des données.

```
class EvaluationVerdict(BaseModel):
    grade: Literal["OUI", "NON"] = Field(description="Verdict final")
    critique: str = Field(description="Pourquoi la réponse est refusée")
```

Note : À ce point il faut comprendre que propager la totalité des messages vers l'évaluateur peut le rendre fou. En effet, si on comprend que toute la logique est bouclée, alors les messages vont s'empiler les uns sur les autres. Si le juge reçoit une trop grande quantité de message il risque de s'y perdre

Le juge aura l'allure suivante:

```
evaluator_llm = model.with_structured_output(EvaluationVerdict)

def quality_control_node(state: MyGraphState):
    """Audits the agent's response based on tool evidence."""

    messages = state["messages"]
    last_agent_message = messages[-1].content
    audit_prompt = SystemMessage(content=f"""
    You are a Neutral Quality Auditor.
    Your ONLY source of truth is the message history (Tool outputs).

    ### AUDIT STEPS:
    1. **Fact Check**: Identify the ...
    2. **Policy Check**: Identify the ...
    3. **Logic Check**: Compare ...
    4. **Verification**:
    - Does the Agent's final answer match these specific tool results?
    - Is the calculation of ... correct?

    ### VERDICT:
    - Grade "OUI" only if ...
    - Grade "NON" if the Agent ignores ...

    ### AGENT OUTPUT TO AUDIT:    "{last_agent_message}"    """)
    verdict = evaluator_llm.invoke([audit_prompt] + messages)
    return {"verdict": verdict}
```

Les fonctions **should_continue** et **route_after_eval** sont assez standard:

```
def should_continue(state: MessagesState):
    last_message = state["messages"][-1]
    if last_message.tool_calls:
        return "tools"
    return "evaluator"
```

La fonction **should_continue** regarde si un outil doit être utilisé, en quel cas elle redirige vers les outils, sinon elle redirige vers l'évaluateur.

```
from langgraph.graph import END

def route_after_eval(state: MyGraphState):
    """
    Fonction de routage conditionnel :
    Analyse le verdict de l'évaluateur contenu dans le 'state'.
    Si le verdict est 'OUI', on dirige vers la fin (END).
    Si le verdict est 'NON', on renvoie vers l'agent pour une correction.
    """

    verdict = state.get("verdict")
    if verdict and verdict.grade == "OUI":
        return END
    return "agent"
```

La fonction **route_after_eval** fait arrêter ou continuer la boucle en fonction du verdict.

Exercice

Créer un agent SAV avec un évaluateur qui soit capable de lire les données de la base de données “[boutique.db](#)” pour un utilisateur et une commande donnée, calculer la différence entre la date de la commande et la date actuelle. Et décider en fonction de la catégorie du produit (pseudo RAG) si le produit peut être retourné.

```
def query_knowledge_base(query: str):
    return "Electronics: 14 days, sealed package. Furniture: 30 days, -15% fee if assembled. Appliances: 30 days, 20€ flat fee."
```

TBD

Human-in-the-loop (Validation Humaine)

La Persistance et la Mémoire (Checkpoints) ainsi que le Time Travel

Multi-Agent & Orchestration (Deep Agents)

Spécialisation avec LoRA (Low-Rank Adaptation)

Les Connecteurs (Toolkits)

deploiement **LangServe**